LeoWay - System Specification

Version 1.0 · 22 October 2026

Team: LRD (Leo Research Department)

Authors: Laurenz (LPuehringerStudent), Ayan (Ayan2310), David (David-Fruehwirt), Behrang (BehrangEl11).

1. Initial situation and goal

1.1 Application domain

LeoWay belongs to the area of school commuting by public transport. It is built for students at HTL Leonding who travel by bus, tram or train, and for form teachers who deal with late arrivals. The student council uses the collected transport statistics to discuss unreliable connections.

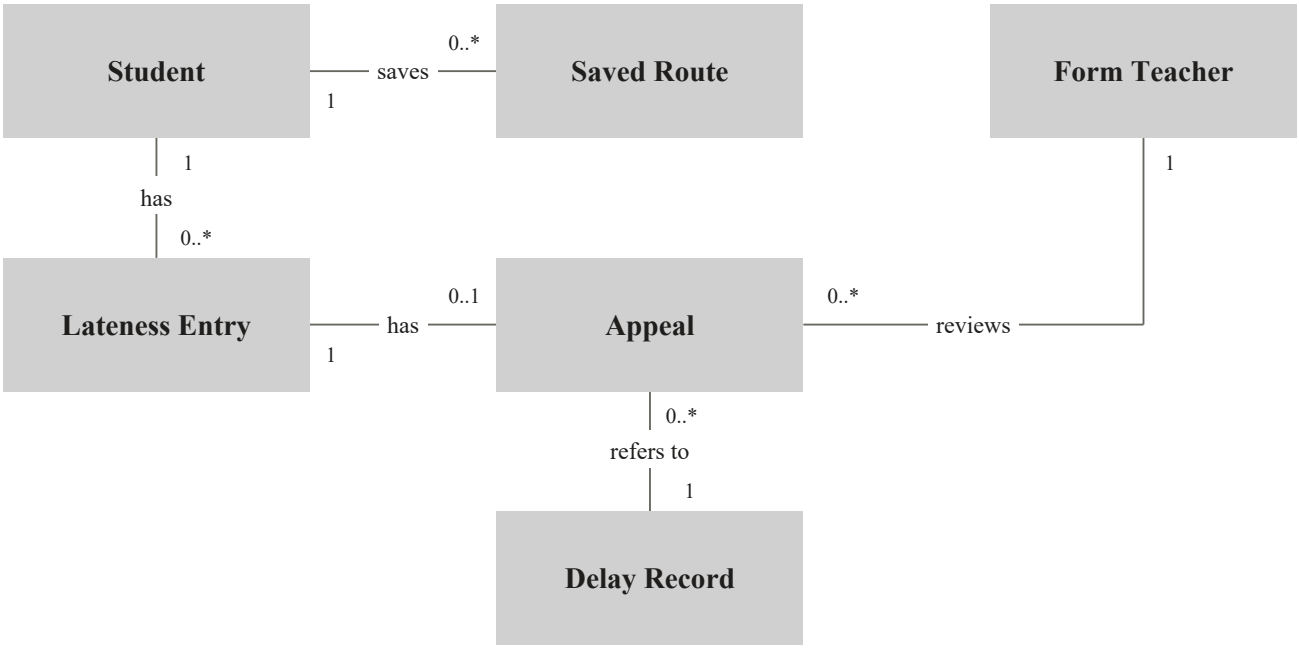
The school is served by Linz AG bus 19 at Limesstraße, trams 3 and 4 and buses 11, 19 and 191 at Meixnerkreuzung, regional OÖVV buses and ÖBB trains. Lessons start at 07:05 or 08:00. A student's actual first and last lessons can change because of cancellations and substitutions.

The school plans lessons in WebUntis, including cancellations, substitutions and exams. Teachers record late arrivals there as an absence; the form teacher decides whether it is excused. Transport sources provide timetables and, where available, live departures and delays. LeoWay connects these two areas: the student's school timetable and their journey to and from school.

The main activities are planning when to leave, reacting to a delayed connection, checking a delay as evidence for lateness and collecting statistics. Students choose their route and alert channels. Teachers check the transport evidence and make the decision about an excuse themselves. LeoWay does not operate transport services or change records in WebUntis. Auspex is an internal backend technology, not a tool for students or teachers.

Application domain diagram

1 means one, *0..1* means zero or one, and *0..** means zero or more.



A student can save several routes and have several lateness entries. A lateness entry can have an appeal, which the form teacher reviews. Each appeal refers to one recorded delay; the same delay can be used by several students. Auspex is a technical backend component, so it is not part of this domain diagram.

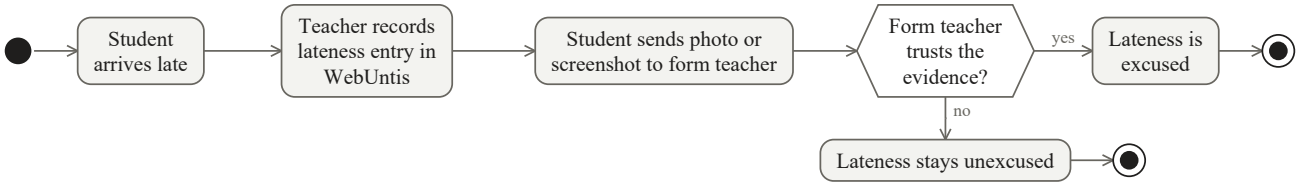
1.2 Terms

Term	Meaning
Access key	The student's WebUntis key (Profil → Freigaben), used instead of the password.
Form teacher	The class teacher (Klassenvorstand) who decides about excusing lateness.
First real lesson	The first lesson that still takes place after cancellations and changes.
Exam day	A day with an exam (e.g. a Schularbeit) from WebUntis. Manual marking is a fallback when exam data is missing.
Leave-by time	When to leave home to reach the first real lesson, including walking time and a buffer.
Lateness entry	A WebUntis absence starting at the first real lesson and ending within it; longer absences are not matched automatically.
Delay record	A stored departure with its line, stop, planned time, reported delay, source and collection time.
Delay appeal	An email from the student to the form teacher that links a lateness entry to a delay record.
Verification link	A link in the appeal showing only the shared delay record.
Crowd report	A disruption reported by a student; several reports can trigger a warning.
Auspex	A programming language used only inside the backend. Students and teachers cannot write, upload or run Auspex code.

1.3 Business processes today

BP1 · Getting to school. The student checks WebUntis for changes and, separately, Scotty or the Linz AG app for departures, and works out the leave-by time in their head. Delays are usually noticed at the stop, too late for an earlier connection.

BP2 · Excusing lateness. The teacher in the first lesson records a lateness entry in WebUntis. The student sends a photo of the vehicle display or a Scotty screenshot to the form teacher, who cannot check whether it is genuine or from that morning.



BP3 · Reporting problem connections. Nobody collects evidence about connections that are often late, so the student council has nothing to take to Linz AG or OÖVV.

1.4 Goal

Goal	Target
Timetable-aware alerts	Every alert uses the student's first real lesson, including cancellations, substitutions and exams (BP1).
Early delay warnings	A student whose saved journey will arrive late is warned and offered another connection where one exists (BP1).
Trustworthy appeals	A late student prepares an appeal in under one minute; the form teacher checks it through the verification link (BP2).
Data history	The delay recorder runs continuously from the first sprint, including holidays (BP3).
Crowd reports	A disruption confirmed by several students triggers a warning, often before official feeds show it.
Prediction (optional)	If we add a model, compare it with "same delay as yesterday" and "average delay of this trip". The main app works without it.
School statistics	One report on connections that regularly arrive after lessons start, handed to the student council (BP3).

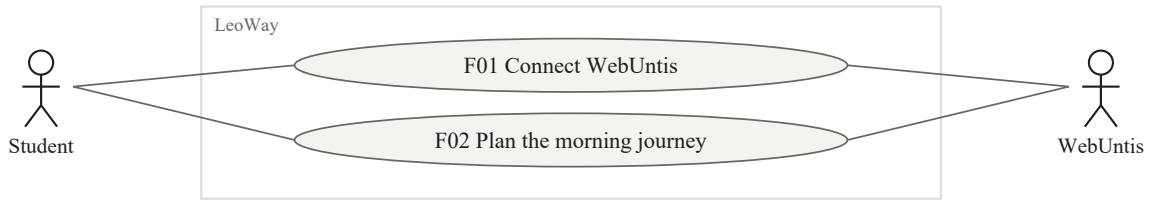
Limits: LeoWay only reads WebUntis. Students send appeals themselves and teachers still decide; a delay record shows what a source reported, not that the student was on the vehicle. Not included: GPS, home addresses and a native iPhone app (iPhone users use the website). Other schools, indoor navigation and a public delay dataset are later extensions.

2. Functional requirements

The functional requirements are grouped into four blocks. Each block starts with a use-case overview diagram. User use cases list the process, goal, precondition, postcondition, roles and trigger, followed by normal and error scenarios, screen sketches and an activity diagram. F06 describes the internal backend integration and has no user screen. Values in the screen sketches illustrate the scenarios.

2.1 Block A: Timetable and leave-by time

The student connects WebUntis once and saves a route; LeoWay then calculates the leave-by time every school day.

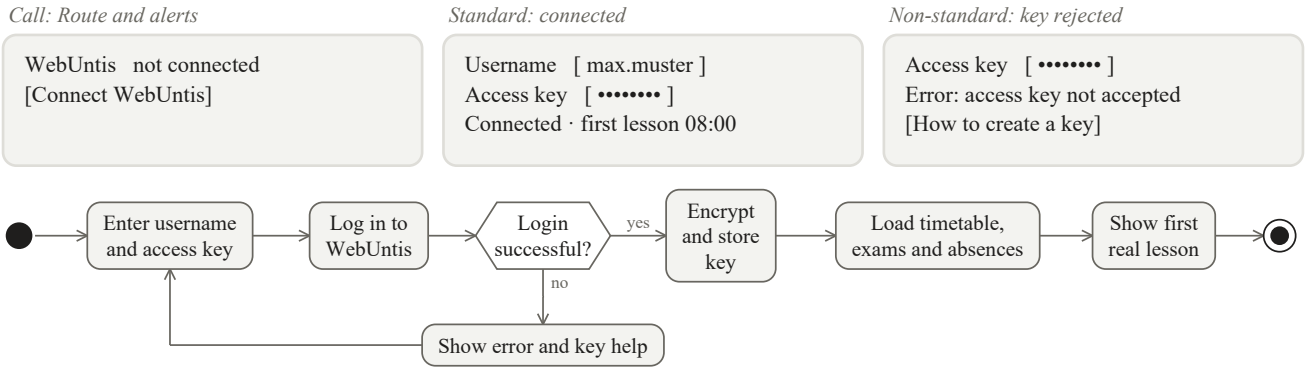


F01 · Connect WebUntis

Business process	BP1 Getting to school, BP2 Excusing lateness	**Goal**	LeoWay can read the student's lessons, exams and lateness entries.
Precondition	The student has a LeoWay account and a WebUntis access key.	**Postcondition**	The key is stored encrypted; the first sync has loaded the timetable. Absence reasons and notes are never stored.
Involved roles	Student; WebUntis (external system)	**Triggering event**	The student taps "Connect WebUntis" during setup or in Route and alerts.

Standard scenario. The student enters username and access key. LeoWay logs in, loads the timetable and shows the next first real lesson.

Non-standard scenario. The key is wrong or revoked: LeoWay stores nothing and shows "Access key not accepted" with help. If WebUntis is down, the student retries later.



F02 · Plan the morning journey

Business process	BP1 Getting to school	**Goal**	The student knows when to leave home for the first real lesson.
Precondition	WebUntis is connected (F01).	**Postcondition**	A leave-by time exists for the next school day and is updated when lessons change.
Involved roles	Student; WebUntis	**Triggering event**	The student saves a route, or a timetable sync finds a changed first lesson.

Standard scenario. The student picks home and school stops, walking times and a buffer. LeoWay shows the leave-by time for the latest fitting connection.

Non-standard scenario. The first lesson is cancelled overnight: the next sync moves the leave-by time and the phone alarm (F04). If no connection fits, LeoWay shows the earliest possible arrival.

Call: Route and alerts

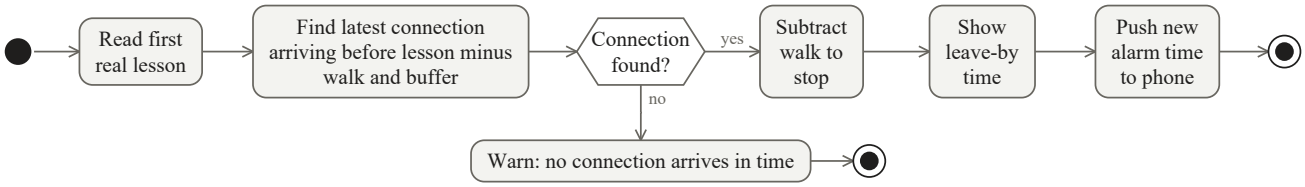
Home stop Leonding Zentrum
School stop Meixnerkreuzung
[Edit route]

Standard: leave-by time

First lesson 08:00
Bus 19 07:31 → 07:47
Leave by 07:24

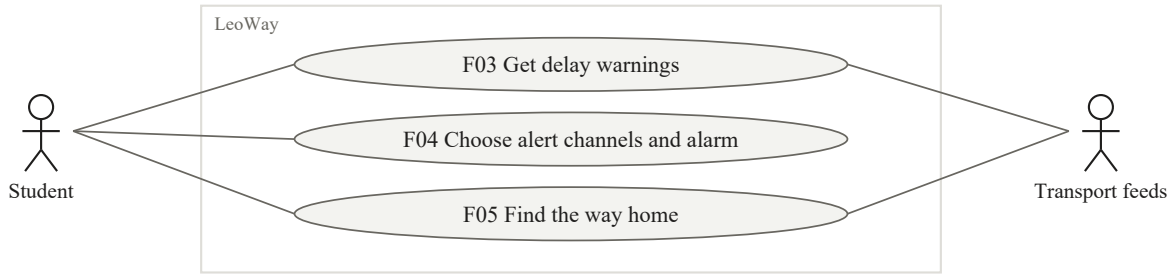
Non-standard: lesson cancelled

First lesson 08:55 (08:00 cancelled)
Leave by 08:16 (was 07:24)
Alarm moved



2.2 Block B: Alerts

LeoWay watches the student's connection and alerts them in the channels they chose.



F03 · Get delay warnings

Business process	BP1 Getting to school	**Goal**	The student learns early that the saved journey will arrive late and gets an alternative.
Precondition	A leave-by time exists (F02); at least one alert channel is set (F04).	**Postcondition**	The student has received one warning per channel for the change; the alternative is shown on Home.
Involved roles	Student; transport feeds	**Triggering event**	A feed reports a delay or cancellation on the student's connection.

Standard scenario. Bus 19 is +6 min, so the student would arrive 3 minutes late. LeoWay warns and suggests tram 3, adding an exam note if the first lesson is an exam.

Non-standard scenario. Without live data for the line, LeoWay shows "timetable only" and the last update time, and sends no delay warning.

Call: Home

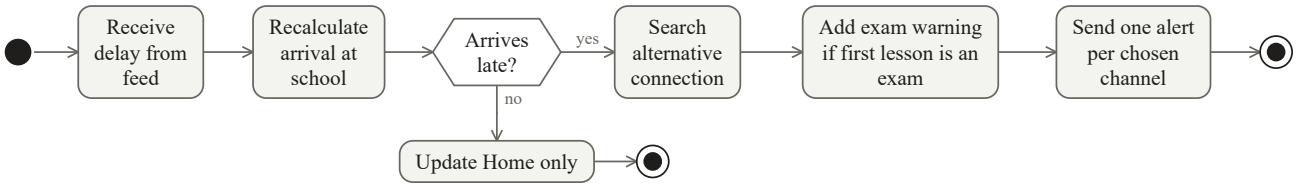
Bus 19 +6 min
[View another connection]

Standard: warning

Warning: Bus 19 +6 min, arrive 08:03
Exam in first lesson
Take tram 3 at 07:29 instead

Non-standard: no live data

Bus 611 07:10
Timetable only
Last update 05:41



F04 · Choose alert channels and alarm

Business process	BP1 Getting to school	**Goal**	Alerts reach the student where they look; on Android, a leave-by alarm rings.
Precondition	The student is logged in. For the alarm: the Android app is installed.	**Postcondition**	Chosen channels are stored; the Android alarm is set for the current leave-by time.
Involved roles	Student; Discord, push service, email	**Triggering event**	The student opens "Route and alerts".

Standard scenario. The student picks Discord and push and gets a test message on each. On Android, the alarm is set for the leave-by time and follows its changes. Channels are built in the order Discord, push, email.

Non-standard scenario. Android denies the exact-alarm permission. The app explains why it is needed and offers a normal high-priority notification instead.

Call: Route and alerts

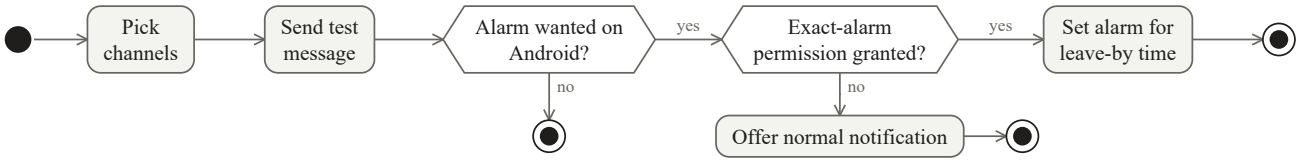
Alerts [Choose channels]
Alarm off

Standard: channels set

[x] Discord [x] Push [ ] Email
Alarm on · 07:24
Test message sent

Non-standard: no permission

Alarm needs "exact alarms"
[Open settings]
[Use notification instead]



F05 · Find the way home

Business process	BP1 Getting to school (return trip)	**Goal**	The student sees the next connection home after the actual last lesson.
Precondition	WebUntis is connected and a route is saved.	**Postcondition**	Home shows the next journey home.
Involved roles	Student; WebUntis; transport feeds	**Triggering event**	The last lesson of the day ends, or a sync finds that lessons end early.

Standard scenario. The last lesson ends at 15:25. LeoWay shows tram 3 at 15:34, allowing the walk to the stop.

Non-standard scenario. The last two lessons are cancelled. The next sync moves the journey home to the new end time, 13:45, and shows the next connection after it.

Call: Home (afternoon)

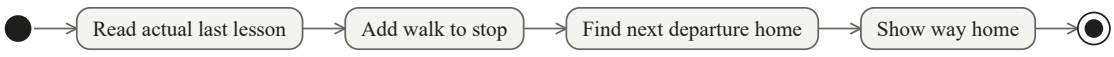
Way home [Show]

Standard: way home

Last lesson ends 15:25
Tram 3 15:34 → 15:52

Non-standard: early finish

Last lesson ends 13:45 (changed)
Bus 19 13:51 → 14:06



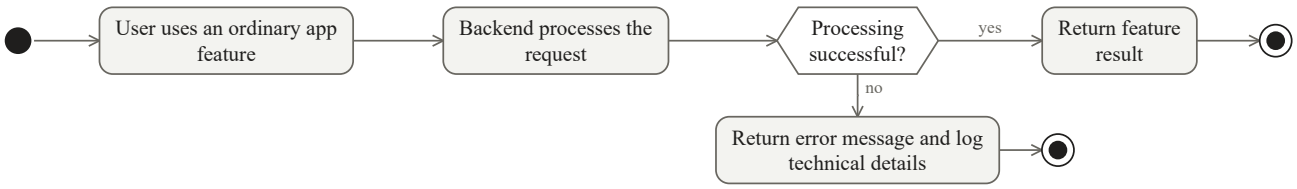
F06 · Internal Auspex integration

Auspex is used only inside the backend. The development team writes and maintains this code as part of LeoWay. Students and teachers use the normal app features; they have no Auspex editor, script upload, execution endpoint or direct access to its runtime.

This is a technical requirement supporting the user features, not a separate student use case. It therefore has no user screen. The exact backend tasks implemented with Auspex and the connection to Quarkus are implementation details to be defined by the team; this specification does not add a student scripting feature.

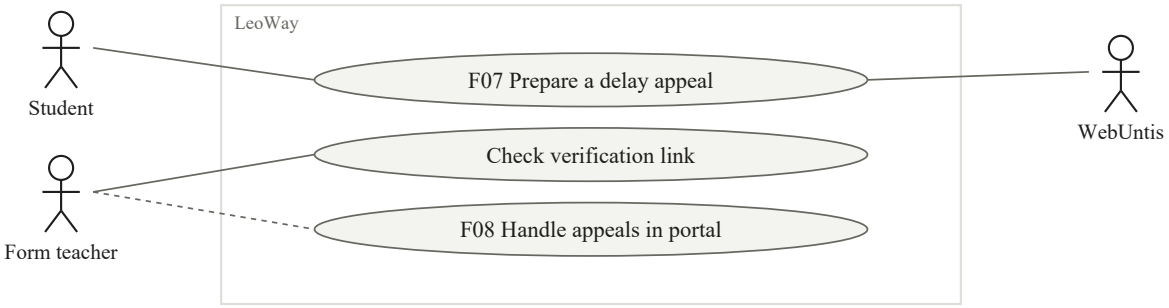
Normal operation. A user uses an ordinary LeoWay feature. The backend performs the required processing and returns the result through the normal API. Any internal Auspex code is maintained and executed by the backend.

Error handling. An internal processing error must not expose code, credentials or server details to the user. The backend returns an appropriate error message and logs technical details for the team without private school data.



2.3 Block C: Delay appeals

LeoWay matches lateness entries with delay records; the form teacher checks the appeal by link or in the optional portal.

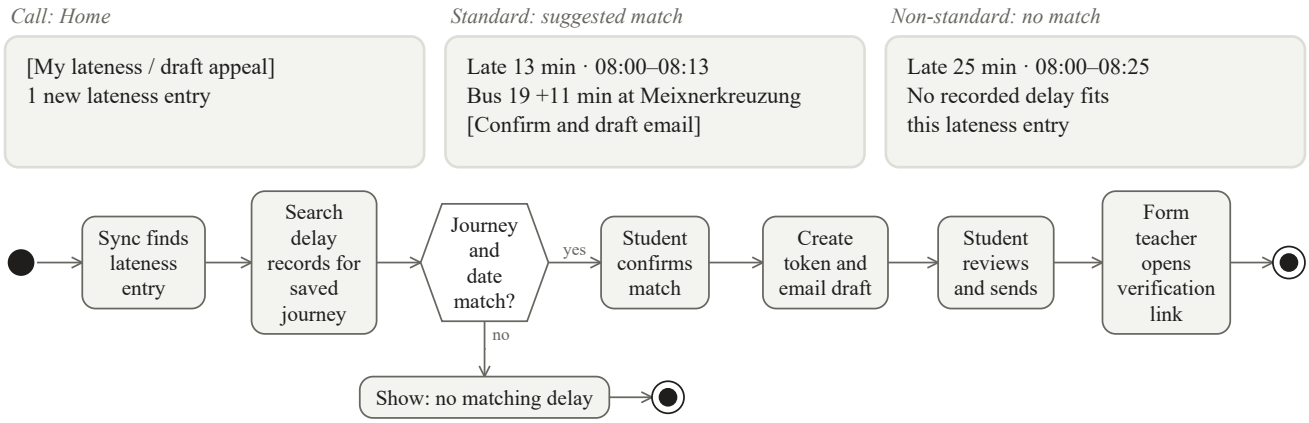


F07 · Prepare a delay appeal

Business process	BP2 Excusing lateness	**Goal**	The student sends a checkable appeal in under one minute; the form teacher can verify it.
Precondition	A lateness entry was synced from WebUntis; a delay record exists for the saved journey.	**Postcondition**	An appeal with a verification token exists; the email is opened in the student's mail app.
Involved roles	Student; form teacher; WebUntis	**Triggering event**	The noon or afternoon sync finds a new lateness entry.

Standard scenario. LeoWay shows "You were marked 13 min late; your bus 19 was 11 min late." The student confirms, checks the editable email (line, stop, times, minutes late, verification link) and sends it. The form teacher opens the link and sees only the delay record.

Non-standard scenario. No record matches the saved journey and date or the absence is longer than the first lesson: no appeal is suggested. Changed or revoked links show "Link not valid".

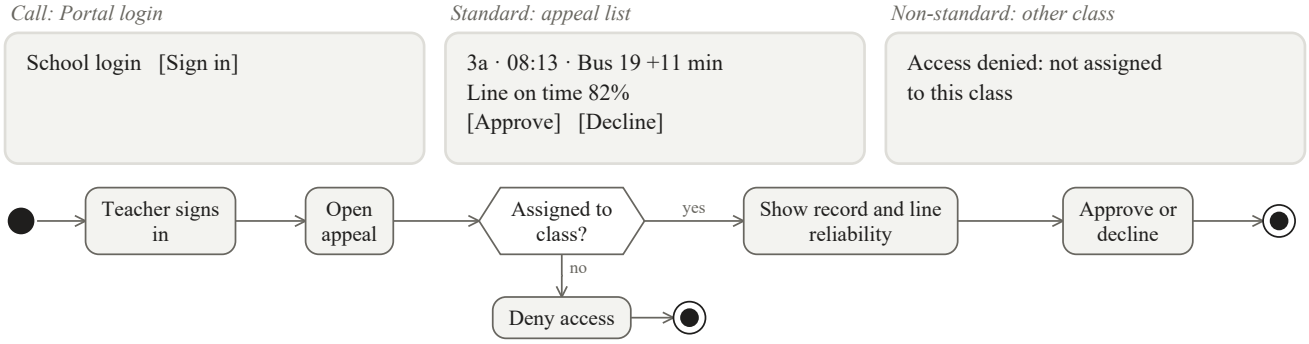


F08 · Handle appeals in the teacher portal (optional)

Business process	BP2 Excusing lateness	**Goal**	Enrolled form teachers handle appeals in one list instead of by email.
Precondition	The teacher is enrolled with a teacher login and assigned to the class.	**Postcondition**	The appeal is marked approved or declined. The teacher still records the decision in WebUntis.
Involved roles	Form teacher	**Triggering event**	A student of the teacher's class sends an appeal.

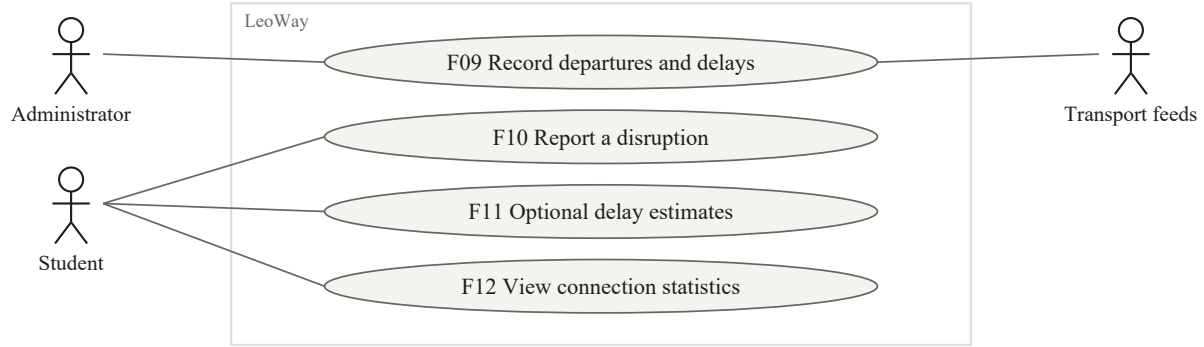
Standard scenario. The teacher opens the portal, sees the appeal with the line's reliability next to it and approves it.

Non-standard scenario. The teacher tries to open an appeal from another class. The portal denies access. Teachers who do not enrol keep receiving emails.



2.4 Block D: Delay data and community

Recorded delays feed warnings, appeals, prediction and statistics; students add crowd reports.

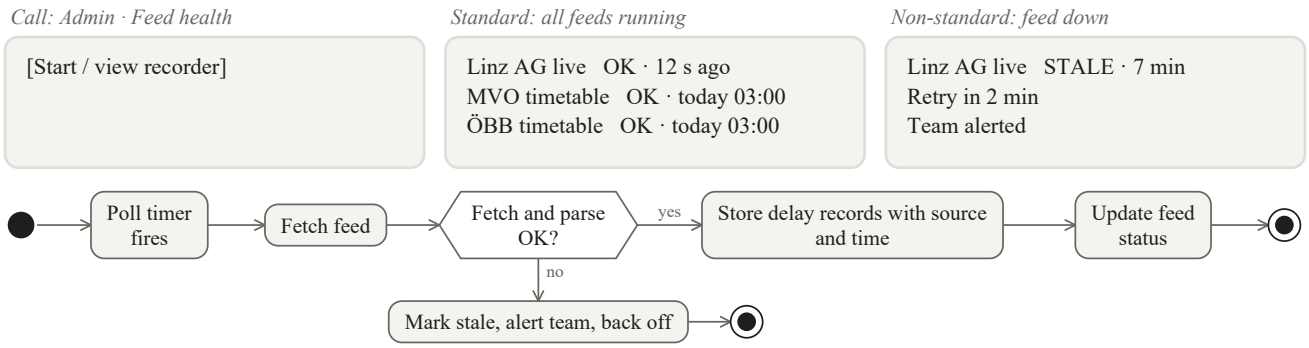


F09 · Record departures and delays

Business process	BP2 Excusing lateness, BP3 Reporting problem connections	**Goal**	A complete delay history with source and collection time.
Precondition	Feed adapters are configured.	**Postcondition**	New delay records are stored; the feed status shows the last update.
Involved roles	Administrator; transport feeds	**Triggering event**	The recorder starts, or a feed poll timer fires, also on holidays.

Standard scenario. The recorder fetches Linz AG departures, stores each departure with line, stop, planned time, delay, source and collection time, and updates the feed status.

Non-standard scenario. A feed fails: other feeds keep running. Show the failed source and last update time, alert the team and retry later. Cached values are labelled as old.

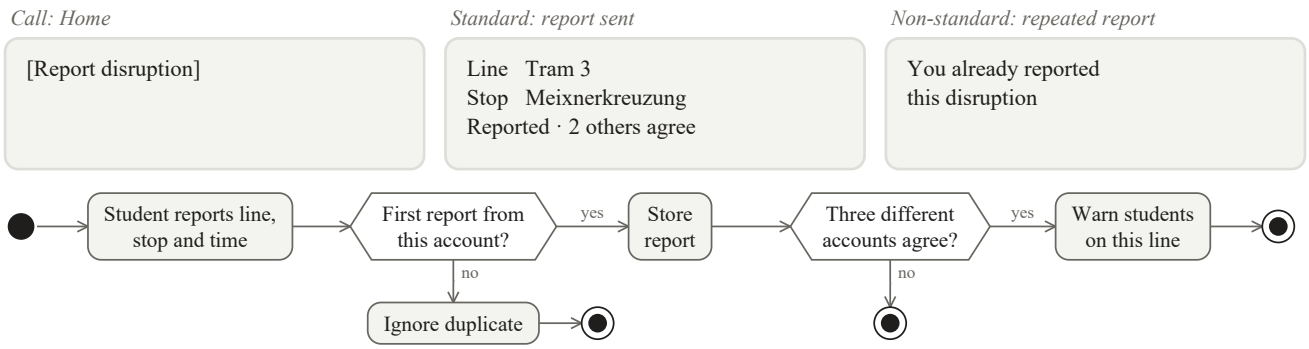


F10 · Report a disruption

Business process	BP1 Getting to school	**Goal**	Students warn each other before official feeds show a disruption.
Precondition	The student is logged in.	**Postcondition**	The report is stored; if the threshold is reached, a warning goes to students using that line.
Involved roles	Student	**Triggering event**	The student taps "Report disruption".

Standard scenario. Three accounts report tram 3 at Meixnerkreuzung for the same current disruption. LeoWay warns students on that line, marked as a crowd report.

Non-standard scenario. One student reports the same disruption several times. The reports count once and no warning is sent.

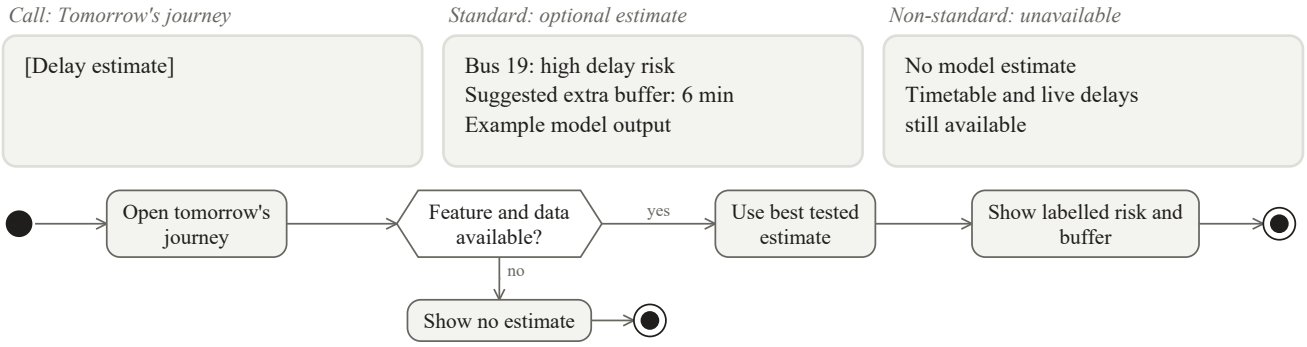


F11 · Delay predictions (optional)

Business process	BP1 Getting to school	**Goal**	Suggest delay risk and an extra buffer, if a model is added.
Precondition	The team has chosen to add this feature and enough history exists.	**Postcondition**	Show a labelled estimate, or keep the normal live-data view.
Involved roles	Student; optional prediction service	**Triggering event**	The student opens tomorrow's journey with the feature enabled.

Standard scenario. If we build a model, it uses recorded history to estimate the risk for tomorrow and suggest a buffer. Compare it with "same delay as yesterday" and "average delay of this trip" on later, unseen data. Python is optional if we choose it for training.

Non-standard scenario. The feature is not included or there is too little history: show no model estimate. If a trained model loses to a simple method, use the better method and label it. Normal timetable and live-delay alerts still work.

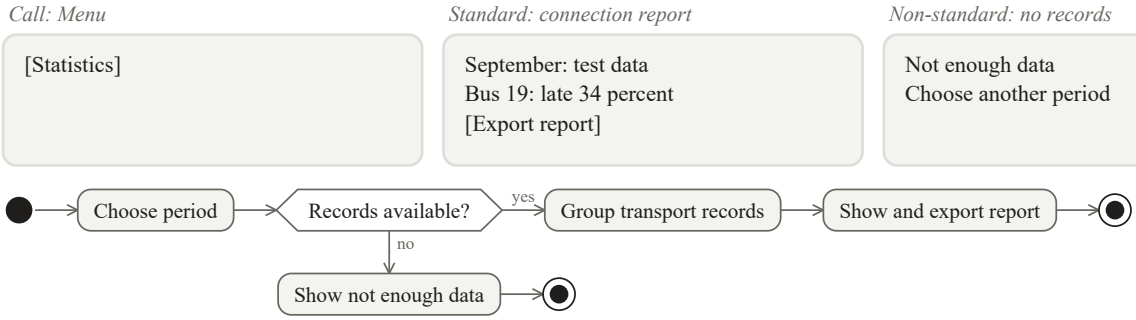


F12 · View connection statistics

Business process	BP3 Reporting problem connections	**Goal**	Give the student council evidence about unreliable connections.
Precondition	Recorded transport data exists for the selected period.	**Postcondition**	Show a connection report or explain that data is missing.
Involved roles	Student or student council member	**Triggering event**	The user opens Statistics and selects a period.

Standard scenario. Select a school month. The app lists connections that often arrive after lessons start and lets the user export a report. It uses transport records, not named student attendance.

Non-standard scenario. No records exist for the selected period: show "Not enough data" instead of a misleading percentage. No student names appear in the report.



3. Non-functional requirements

3.1 Usability

Our main users are students checking their phone before or during the journey and form teachers checking an email. The student view uses a departure-board layout: first lesson, leave-by time, line and delay appear together on a phone-width screen without opening another page. Verification links open in the teacher's browser without an account.

Use plain messages, readable labels and keyboard-accessible controls. Check Chrome, Firefox and Safari, a narrow phone screen and the Android app on a real phone. Preparing an appeal from a suggested match must take less than one minute.

3.2 Efficiency

These are first test targets, not measured results or confirmed school-wide capacity. Check them after the first prototype and revise them with the team if needed.

ID	Proposed target and check
E01	With 10 test users active together, 95 percent of cached journey requests finish within two seconds on the test server. Record the response times.
E02	Evaluate a newly received delay within 60 seconds. Send the same change only once per chosen channel. Measure channel delivery separately from backend processing.

Provider update rates can limit how early we detect a delay. Always show the source update time. A failed live feed is marked unavailable; cached values stay clearly labelled as old.

3.3 Maintainability

Another group may continue the project. Keep setup steps, the API description and tests in the repository. Review changes and run backend/frontend checks in CI.

Each source has one adapter, and alert channels share one interface. Adding another transport feed or notification channel should not require rewriting the recorder or other adapters. Keep the optional model separate so the main system works without it.

3.4 Security

Confidentiality: use HTTPS, encrypt WebUntis keys and keep the encryption key outside the repository and database. Never log keys or private school data. Check record ownership on every private request. Teachers in the optional portal only see assigned students; the normal admin view shows feed status, not private records.

Data integrity: students cannot change collected transport times. Keep the source, trip, stop, service date and collection time. A student confirms a match but does not edit its evidence. Reject changed or revoked verification tokens.

Availability: a failed adapter must not stop the others. Retry failed requests without flooding the source and notify the team if the recorder stops. Keep backups and test a restore on the selected host. Exact backup settings are decided during deployment.

Auspex: only the backend runs code maintained by the development team. Students and teachers cannot submit or execute code. Do not expose the Auspex runtime directly through the website, Android app or public API.

3.5 Legal and school constraints

Personal-data handling must follow the GDPR and Austrian Data Protection Act (DSG). Keep only data needed for the feature and provide a short privacy notice explaining its purpose, storage and deletion. The Austrian Data Protection Authority explains the framework in its privacy overview.

Do not store GPS, home addresses, WebUntis passwords, absence reasons or notes. Use is voluntary. Keep routes and contact details until account deletion; replace timetable caches on sync. Delete appeals and linked lateness entries at school-year end or account deletion. Keep transport history separate. Deleted accounts must stay deleted after a backup restore.

The form teacher decides whether lateness is excused. LeoWay only reads WebUntis. Follow each provider's data licence, attribution, request limits and storage conditions. Use VAO only with active access under its contract; the system must also work without VAO. There is no native iPhone app; iPhone users use the website.

4. Quantity structure

The following is a first sizing example for two classes of 30 students each. These are planning assumptions, not actual class sizes, measured feed volumes or a rollout promise. Replace them with survey and feed measurements during the first sprints.

Quantity	First estimate	Basis
Student accounts	60	2 classes x 30 students
Saved routes	Up to 120	Up to 2 routes per student
Stop master data	Working budget: 1,000 stops	Count the actual records in the selected timetable imports; only part of them is watched live
Stops watched live	10	Example set covering common class routes and school stops
Recorded departures per day	About 2,000	10 stops x 200 departures, assuming one stored observation per departure
Raw delay storage per year	About 365 MB before live updates, indexes and backups	2,000 observations/day x 365 days x an estimated 0.5 kB per observation
Alerts per school day	About 240 channel messages	60 students x 2 alerts x 2 chosen channels
Alert payload per day	About 240 kB, plus network overhead	Estimated 1 kB per channel message
Journey-view JSON	Initial budget: 50 kB per request	10 simultaneous view loads transfer about 500 kB, excluding app files

Saving several delay updates for one departure increases the record count and storage above this base estimate. Measure that update factor with the live-feed spike. WebUntis data is per student. Cache shared transport results; actual request counts depend on source rates and the final sync frequency.

The prediction model is optional and not chosen, so its parameter count, training memory and storage are not estimated yet. Add these only if the team decides to build it. The recorder still collects useful history for appeals and statistics.

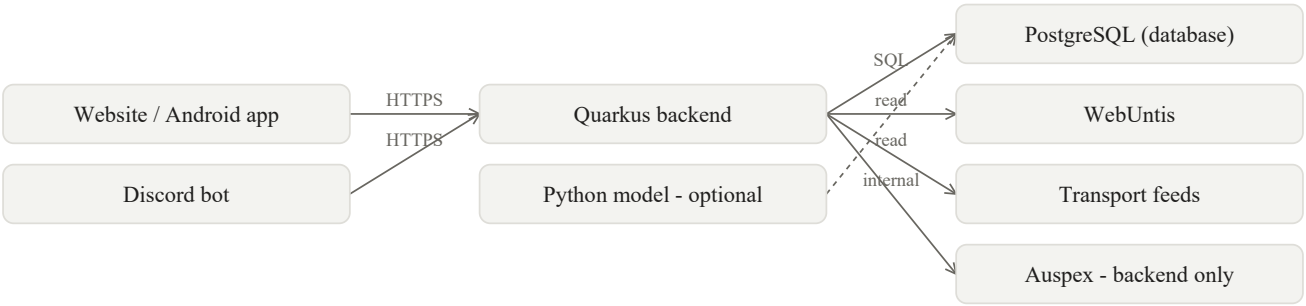
Resources: team computers, a continuously running backend/database server and an Android test phone. No fixed server RAM or CPU claim is made before testing. Use the request and storage

measurements above to choose the final host and backup capacity.

5. System architecture and interfaces

5.1 Main parts

Angular provides the website and teacher portal; Capacitor wraps the app for Android. Quarkus handles journeys, the recorder, appeals and alerts. PostgreSQL stores data. Auspex is used only inside the backend, with code maintained by the development team. Python is optional if we add model training. We use free services and team hardware or school hosting.



The clients communicate with the Quarkus backend and do not connect directly to PostgreSQL or Auspex. Auspex is an internal part of the backend. Its code and integration are maintained by the development team. The exact connection between Quarkus and Auspex must be documented during implementation; no specific connector or runtime protocol is assumed here.

5.2 Communication and external systems

Partner	Interface and exchanged data
Website / Android	REST/JSON over HTTPS: saved routes, lessons, journey results, reports and appeal drafts. Responses include last-update time.
PostgreSQL	Backend SQL connection for routes, observations and appeals. Clients have no direct connection.
WebUntis	Read-only calls through the webuntis adapter using the student's username and access key. Allow login/logout, timetable, timegrid, absences and exams only.
Linz AG	Adapter for live departures and delays. The spike checks EFA or GTFS-RT availability, trip IDs, update frequency and source limits.
MVO and OeBB	GTFS timetable files imported into a common stop/trip format. They provide scheduled times where live data is missing.
VAO	Optional regional source under its active contract. Keep it disabled while usable access is unavailable; use timetables instead.
Discord / push / email	One notification interface with a shared event ID. Discord bot messages, device pushes and email alerts carry the line, delay and relevant journey. Appeal emails open in the student's own mail app.
Auspex	Internal backend integration only. No interface for student or teacher code. Document the Quarkus connection and exchanged data with the implementation.
Optional model / weather	History is passed to training only if the model is built. GeoSphere weather can be an optional input.
Form teacher	Browser verification page showing the chosen transport record. The optional portal needs an authenticated teacher assigned to the class.

5.3 Refresh and time rules

Read lessons and exams in the evening and before/during the morning commute. Check absences again at noon and later in the afternoon, because teachers may enter lateness later. Students can refresh manually. Keep the polling intervals configurable and within source limits.

When the first real lesson changes, push a versioned alarm update to Android. Replace the old alarm and ignore older versions. Sync again when the app opens; offline or force-stopped phones may receive updates late. If exact alarms are unavailable, offer a normal notification.

Choose a journey arriving before the first real lesson, including the school walk and a buffer. Subtract the walk to the departure stop for leave-by time. Match a lateness entry by saved journey and service date; longer absences are not automatically treated as late arrivals, and the student confirms the match. If WebUntis cannot be used, provide timetable entry or an iCal import as a fallback.

Store timestamps in UTC and show Europe/Vienna local time. Keep the service date separately for overnight trips. Label timetable, predicted and confirmed times correctly. A delay record never proves the student travelled on the vehicle.

6. Acceptance criteria

Accept the user use cases against their normal and non-standard scenarios below. Check the internal requirement F06 through backend and access tests. Use recorded or synthetic data for private records. Keep the test results with the delivered version. The model and teacher portal do not block delivery if the team leaves them out.

Test	Pass condition
A01 - F01	A valid key loads lessons, exams automatically and later lateness entries. A wrong/revoked key shows an error without saving it as working. Forbidden data calls are blocked.
A02 - F02	A route gives the correct leave-by time. A cancelled first lesson changes the time and alarm. No fitting journey gives a warning. Check summer/winter time.
A03 - F03	A late journey creates an alert, an available alternative and an exam note when needed. Without live data, show timetable-only status and no invented live delay.
A04 - F04	Each chosen channel gets a test message once. A real Android phone rings at leave-by time and moves the alarm after an update. Without exact-alarm access, offer a notification.
A05 - F05	Show a connection after the last lesson and the walk to the stop. Cancel the last lessons and show an earlier journey home.
A06 - F06	Confirm that Auspex is reachable only through internal backend processing. Neither students nor teachers can submit or execute Auspex code through the UI or API. Internal errors do not expose code or credentials.
A07 - F07	A suggested match opens the correct editable email in under one minute. The link shows the same source record. No match gives no suggested appeal; invalid or revoked links are rejected.
A08 - F08	If built: an assigned teacher sees evidence and can approve/decline. An unassigned teacher is denied; WebUntis is never changed.
A09 - F09	Records keep trip, stop, source, service date and time. A failed feed is marked unavailable while others continue. Retrying does not duplicate observations.
A10 - F10	Three different accounts confirming the same current disruption trigger a crowd-labelled warning. A repeated report from one account is ignored.
A11 - F11	Only if built: compare the model with both simple methods on later data excluded from training. Use the better method if the model loses. With no model/history, normal live alerts still work.
A12 - F12	A known unreliable connection appears in an export without student names. An empty period gives an insufficient-data message instead of a percentage.
A13 - Security	One student cannot read another's data. Check teacher assignments, token revocation, account deletion and a restore that does not bring deleted accounts back.
A14 - Quality	Measure E01/E02, test keyboard and phone-width use, supported browsers, the Android app and CI. Record actual payload/storage sizes and update the quantity estimates.

6.1 Plan until January 2028

The project runs from October 2026 to January 2028, with two-week sprints during school weeks. **Behrang Elyasi is the product owner.** We review this working plan after each sprint and keep final delivery in January 2028.

Period	Planned result
Oct-Nov 2026	Set up the projects, test the live feed and Android alarm, choose hosting and start the recorder.
Dec 2026-Jan 2027	First working version: connect WebUntis, save a route and send a leave-by alert through Discord.
Feb-Jun 2027	Build the website and Android app, delay warnings, changing alarms and delay appeals. Test with students and a form teacher.
Jul-Aug 2027	The recorder continues during holidays. No regular school sprints are planned.
Sep-Nov 2027	Complete crowd reports, the way home, the internal Auspex integration and statistics. Consider the optional model and portal only if time allows.
Dec 2027-Jan 2028	Test the complete system, fix problems, finish setup steps and the user guide, then deliver the final version in January 2028.

Deliver the code, setup steps, a short user guide and test results. Optional extras must not delay the main features.

6.2 Before we start

Prepare team computers, a server and database, an Android phone, test WebUntis access and a Discord bot. Test the Linz AG feed, changing phone alarms and the Auspex connection early. Survey our classes. Software and data sources are listed in section 5.

6.3 Still to decide

Settle these points before building the affected feature. Keep the answers in the related GitHub issues.

Decision	What we need to settle
Hosting	Team server or LeoCloud? Agree who monitors the recorder and handles backups.
Student login	Choose how students sign into LeoWay and reconnect or disconnect WebUntis.
Auspex	Define which internal backend tasks use it and document its connection to Quarkus. Students and teachers have no direct access.
Appeals	Agree the email format with form teachers before building it. Decide where teacher addresses come from; use checked manual entry until then.
VAO	Define request limits and data retention under the active contract. LeoWay must work without VAO.

Later, choose the final name. Decide on the optional model and teacher portal once the main features work. If the portal is added, agree teacher login with the school.